

Rob The Block: Report

Overall Approach to Implementation:

- We used libGDX's ApplicationAdapter as the core game loop, with SpriteBatch for rendering sprites and a tile grid system for level layout.
- Structure the project using Maven for dependency management with the libGDX 1.12.1 library.

Adjustments and Modifications to Initial Design:

- Adapted the UML class diagram to incorporate libGDX-specific classes, such as extending ApplicationAdapter in the Game class and using libGDX's rendering pipeline instead of custom rendering logic.
- The Position class was added to the design to work alongside libGDX's coordinate system for tracking character and object positions on the tile grid.
- Implemented the Enemy class hierarchy with abstract methods to allow for different behaviors between Dog (chase-based) and Homeowner (patrol-based) enemies while sharing common collision detection logic.
- Modified the scoring system to separate level completion tracking from coin management. This allows players to track progress toward opening the exit door independently from their current coin balance, preventing confusion when collecting rewards but losing coins to punishments.
- Introduced game state management to handle transitions between menus, gameplay, and end screens, improving the overall user experience and game flow.
- Expanded the tile system to support different visual elements like furniture and decorations beyond basic walls and floors, allowing for more detailed and realistic level designs.
- Implemented a two-pass rendering approach where the base environment is drawn first, followed by furniture and objects. This solved visual layering problems with larger objects that span multiple tiles.
- Extended the enemy system to support multiple homeowners patrolling different areas of each level, increasing difficulty and requiring more strategic gameplay.

Project Management and Division of Roles:

For managing the project, we meet as a group, divide tasks, have a mid-way checkpoint and keep track of commits using git.

Task distribution:

- Harsh: Implementing our level classes and main, game and UI classes.
- Damiano: Adapting our UML class diagram structure to incorporate the libGDX library.
- Jeff: Implementing our object classes.
- Arnoosha: Implementing our character classes and enemy classes.

External Libraries:

- libGDX - for rendering, input handling and looping the game
- Maven - used for managing the project lifecycle

Code Quality Measures:

- Following separation of concerns by dividing logic across characters, level, and objects packages.
- Using abstract classes (Enemy, GameObject) to share common behavior across subclasses like Homeowner, Dog, Reward, and Punishment.
- Using an enum (Direction) for movement to avoid hardcoded string values.
- Implemented proper encapsulation in the Burglar class by keeping position, score, and noise level as private attributes with controlled access through public methods.

Biggest Challenges

- Scheduling group meetings and task distribution due to different schedules and availability of the group members.
- Configuring Maven and libGDX in VS Code with a nested project structure inside a group Git repository, requiring a .vscode/settings.json to point to the correct pom.xml.
- Resolving libGDX import paths and getting VS Code's Java extension to properly recognize Maven dependencies.
- Understanding how to structure the Enemy class hierarchy to work with Position from the characters package while avoiding circular dependencies.
- Solving merge conflicts wasn't as bad as we thought since of our difference schedules, but that does not mean they didn't happen.
- Dividing tasks. Dividing the functional aspect of game into equal sized doable tasks was a big challenge as a lot of components worked together and some components were dependent on others.
- Finding sprites turned out to be harder than expected but making sure to keep a consistent theme and finding sprites was challenging.
- Rendering graphics. Not all elements on the game are of the same size we ran into issues where the images would not render properly because we were rendering tile by tile, so we had to switch it to render a base first and then render all the objects on top it to avoid bugs.
- Implementing the dual-scoring system required careful separation between rewards collected (completion metric) and coin score (survival metric) to prevent confusion when punishments reduce score but don't affect progress toward opening the exit door.
- Designing a scoring system that accurately represents both player progress and performance. We needed to balance rewarding exploration with penalizing mistakes while ensuring players could always tell whether they had collected enough items to advance.

- Managing game state and ensuring smooth transitions between different screens (start menu, gameplay, game over and victory). We had to carefully reset variables and handle both keyboard and mouse input to prevent bugs during restarts and level changes.
- Generating levels with random reward placement while ensuring they appear only on valid, accessible tiles. This required adding validation checks during level initialization to avoid rewards spawning inside walls or on furniture.
- Coordinating multiple enemies with different movement patterns without causing performance issues or collision detection bugs. We implemented timer-based updates and patrol path systems to keep enemy behavior consistent and predictable.
- Creating distinct, playable level layouts that feel like actual houses while maintaining balanced difficulty progression. Each level needed to be visually interesting, strategically challenging, and fair to the player.